

Week 0

Course Overview & Handbook

KIUA2003 — Applikasjon av kunstig intelligens / maskinl ring II

Course description, syllabus, project brief, grading rubric, cost & safety guidelines, and reading list for KIUA2003 AgentCom.

CONTENTS

Reading	Syllabus
Reading	Project Brief
Document	Project Brief (PDF Download)
Reading	Grading Rubric
Reading	Cost & Safety
Reading	Reading List & References
Document	Course Handbook (PDF)
Document	Glossary - Agent Communication & Multi-Agent Systems
Discussion	Introduce Yourself & Your Scenario Choice

READING

Syllabus

Syllabus - How LLMs and AI Agents Talk to Each Other

Level: BSc Artificial Intelligence, Year 2 · **Duration:** 4 weeks · **Format:** weekly lecture (≈90 min) + supervised lab (≈120 min) · **Effort:** ~6–8 h/week including project.

Module description

Modern AI systems increasingly consist of **multiple** language models that talk to one another: a model that critiques another model's answer, two agents that debate or negotiate, a "society of mind" that solves a task by discussion. This course teaches how that communication actually works under the hood. Students build, from scratch and in plain Python, a system in which **two LLM agents hold a conversation to accomplish a goal**, bounded by an explicit budget of turns, tokens, or time, and then **analyse what happened** - did the agents converge, how much did it cost in compute and time, and how good was the result?

We deliberately avoid multi-agent frameworks. The goal is to understand the machinery, not to operate a library.

Prerequisites

- Programming in Python (functions, classes, dictionaries, reading/writing files, JSON).
- Introductory ML / AI (what a model is; what training vs. inference means).

Learning outcomes

By the end of the course a student can:

1. **Explain** an LLM as a stateless function over a list of (role, content) messages, and define *token*, *context window*, and *temperature*.
2. **Implement** a conversation loop in which one agent's output becomes another agent's input, with correct per-agent role-mapping.
3. **Design** agent behaviour through system prompts, roles, and personas.
4. **Enforce** termination and resource control: max turns, token budget, wall-clock limit, and goal conditions - and explain why this is essential for multi-agent systems.
5. **Manage** the conversation context when it grows past the window (truncation, summarisation) and justify the choice.
6. **Choose** between free-text and structured (JSON) inter-agent message protocols, and relate them to classical agent communication languages.
7. **Evaluate** a multi-agent dialogue: build a *judge* agent, know its biases, and measure cost, turns-to-convergence, and task success.
8. **Diagnose** multi-agent failure modes: sycophancy, loops, drift, forgetting, hallucination, error propagation, runaway cost, premature termination.

Weekly schedule

Wk	Lecture theme	Lab outcome	Project milestone (due start of next week)
1	LLM as a message function; tokens; the "telephone" idea	First API call → 2-agent ping-pong of N turns	**M1:** Working ping-pong + 1-page design doc
2	Roles & role-mapping; the orchestration loop; termination & budgets	`DialogueEngine` running a full scenario	**M2:** Configurable engine + saved transcripts
3	Context/memory management; structured messages; judge agents	Eval harness + parameter experiments	**M3:** Experiment results comparing ≥3 runs
4	Failure modes; cost/time accounting; reproducibility; frameworks (concept)	Polish + failure analysis + rehearsal	**Final:** Report + video demo

Assessment

This module is graded **pass/fail** as a compulsory assignment (arbeidskrav).

To **pass**, you must:

- Submit your code.
- Complete the weekly quizzes. (doesn't affect the grade)
- Deliver a 2-5 minute video demo and project demo with explanations.

Submission: a PDF report and results of one test session with agent communications, plus a video presentation. Lab work may be done in pairs; the report and demo are assessed individually.

Tools & setup

- Python 3.10+

- Ollama with a small instruct model (e.g. llama3.2:3b or qwen2.5:3b) - fast enough on a laptop, good enough for dialogue.
- The course starter-code/ skeleton (includes a **mock mode** so students without a GPU, or working offline, can still develop everything except final runs).
- Editor of choice; terminal; VS Code.
- Submission on Canvas.

Academic integrity

Using an LLM to help **write code** is allowed and expected - this is a course about LLMs. Students must, however, **understand and be able to explain every line they submit** (there might be random group selection for oral presentations if needed), and must cite any code generated or copied. Two students may pair on lab code but submit individual reports.

READING

Project Brief

Project Brief - A Bounded Two-Agent Dialogue System

You will build **one system across all four weeks** and analyse its behaviour. The same brief applies to everyone; you choose a **scenario** that gives your system its personality.

The goal

Build a program in which two LLM agents hold a conversation to accomplish a goal, under an explicit budget (turns / tokens / time), that logs every message, and that you can analyse afterwards: did the agents reach the goal, how many turns and tokens did it take, and how good was the outcome?

This is intentionally small in scope and deep in understanding. A working system is two agents, a loop, a budget, and a transcript file. Everything else is refinement.

Pick a scenario (choose one in Week 1)

Scenario	Agent A	Agent B	"Goal reached" means...
Debate	Pro side	Con side	A 3rd <i>*judge*</i> agent picks a winner after N rounds
Collaborative solver	Proposer	Critic	They jointly produce a correct solution to a puzzle/task
Interview	Interviewer	Candidate	A rubric is satisfied / N questions asked
Socratic tutor <i>*(Recommended)*</i>	Tutor (never gives the answer)	Student	The student states the correct idea in their own words
Red vs. blue <i>*(Recommended)*</i>	Attacker (finds a flaw)	Defender (patches it)	No new flaw found in a round

You may propose your own scenario - clear it with the instructor in the Week 1 lab.

What "two agents talking" means technically

Each agent is just a **system prompt + the shared conversation history**. The engine:

1. Renders the history as agent A's view and sends it; A produces a message.
2. Appends A's message to the neutral transcript (labelled as A).
3. Renders the same history as agent B's view and sends it; B replies.
4. Appends B's message; repeat - **until a stop condition fires**.

The subtlety students discover: from agent B's point of view, agent A's messages must appear as the **user** and B's own as the **assistant** - and vice-versa. Getting the role-mapping right is half of Week 2.

Milestones

All milestone deliverables are submitted together as a final compulsory assignment at the end of Week 4.

M1

- A script that makes two agents exchange **N turns** and prints the transcript.
- A **hard cap** on the number of turns (no infinite loops - ever).
- A **1-page design doc**: chosen scenario, the two personas (with their full system prompts), what "goal reached" means, and how you'll know.

M2

- A reusable `DialogueEngine` driven by a **config file** (models, personas, budgets).
- **Three independent stop conditions** working: max turns, token budget, wall-clock, with the firing condition recorded.
- Every run saves a **structured transcript** (JSON) with per-message token/time data and a run header.

M3

- Context handling when history exceeds the window (truncate **or** summarise - your choice, justified).
- A **judge / evaluation step** that scores each run.
- An **experiment**: vary one parameter (temperature, persona wording, budget, or model) and compare ≥ 3 runs in a small table.

Final

- Polished repository (README, runnable with one command).
- **Report (4-6 pages)**: design, results table, a **failure-mode analysis** (show a run that went wrong and explain why), and **cost/time accounting**.
- **Video demo (2-5 min)**: record yourself running the system, testing it, and explaining the most important parts of your code.

Stretch goals (optional, for strong students)

- Give an agent a **tool** (a calculator or a search function it can "call" by emitting a special token your loop intercepts).
- Add a **third agent** (e.g. a moderator that decides who speaks next).

- Compare your hand-built loop against a framework (AutoGen/LangGraph) and discuss what the framework adds - and hides.

DOCUMENT

Project Brief (PDF Download)

Attached file: [project-brief-kiua2003.pdf](#)

READING

Grading Rubric

Grading Rubric

This module is graded **pass/fail** as a compulsory assignment. Each milestone and the final submission are reviewed against the criteria below. To pass, your project must meet the **pass** standard for every milestone.

Weekly quizzes (not graded and counts toward your final grade or passing compulsory)

Four quizzes, one per week, each covering that week's material. All ~15 questions are multiple choice, scored out of ~30 points. Taken individually, online at home, open notes, with a ~25-minute limit.

M1 - Design doc & ping-pong

Criterion	Pass	Fail
Two agents exchange N turns and it runs	Loop works, output readable, roles labelled	Does not run or agents don't exchange messages
Hard turn cap present	Cannot loop forever; `Budget` used or equivalent	No turn cap or easily bypassed
Design doc: scenario & personas	Personas described, system prompts included	Missing or too vague to understand the scenario
Design doc: "goal reached" defined	Clear, testable success criterion	No criterion or hand-wavy

M2 - Dialogue engine

Criterion	Pass	Fail
Correct role-mapping between agents	Each agent sees the other as `user`, self as `assistant`	Role-mapping broken or missing
Config-driven (models, personas, budgets)	Key params in a config file, not hard-coded	Everything hard-coded
Three stop conditions (turns / tokens / time)	All three independently enforced	Fewer than three working
Structured transcript with per-message metrics	JSON with role, content, tokens, time per message	No transcript or missing key metrics
Code quality	Readable and reasonably organised	Impossible to follow

M3 - Control, evaluation & experiments

Criterion	Pass	Fail
Context management when window fills	Truncation or summarisation implemented, with justification	No context handling; breaks on long runs
Judge / evaluation step	A judge scores each run with defensible logic	No evaluation step
Experiment varying ≥ 1 parameter	≥ 3 runs compared with a conclusion	No experiment or no comparison
Structured vs. free-text reflection	Tried or argued the choice	Not addressed

Final report

Criterion	Pass	Fail
Design clearly explained	A reader could re-build the system	Major gaps
Results table	Clear metrics, honest reporting	Missing or no real data
Failure-mode analysis	Shows a real failed run, diagnoses the cause	No failure analysis
Cost / time accounting	Tokens & wall-clock measured and discussed	Not measured
Writing & structure	Clear, within 4-6 page length	Disorganised or way off length

Video demo

Criterion	Pass	Fail
System runs	Runs and produces output (fumbling is fine)	Does not run
Explains own code	Can walk through core logic and justify design choices	Cannot explain core logic
Tests the system	Demonstrates the system with real input and discusses output	No testing shown

Overall pass/fail

- **Pass:** All milestones meet the pass criteria above and video demo delivered.
- **Fail:** Any milestone missing or below the pass standard, or inability to explain your own code in the video demo.

READING

Cost & Safety

Cost & Safety One-Pager

You are about to make **two LLMs talk to each other in a loop**. This is the single easiest way for a beginner to start a process that never stops. With local models the cost is not money - it is ****your laptop pinned at 100% generating tokens forever, your context window**

overflowing, and a lab full of frozen machines.** The discipline below is part of the grade.

The three budgets - always all three

Every dialogue loop must be bounded by **all three** of these at once. The provided

Budget class (starter-code/budget.py) enforces them; use it from Week 1.

Budget	Stops when...	Why you need it
Max turns	a fixed number of messages have been exchanged	The fundamental anti-infinite-loop guard
Token budget	total tokens generated exceed a limit	Stops a chatty pair from filling the context window and slowing to a crawl
Wall-clock	the run has taken more than N seconds	Catches a model that hangs or a turn that runs away

A loop with only *one* of these can still misbehave (e.g. max-turns is fine but each turn generates 4 000 tokens). Belt **and** braces.

The rule

No `while` loop that calls a model may exist in your code without a `Budget` controlling it.

In practice:

```
budget = Budget(max_turns=12, max_tokens=4000, max_seconds=120)
while not budget.exhausted():
    reply = client.chat(model, messages, temperature=0.7)
    budget.record(turns=1, tokens=reply.tokens)
    ...
budget.stop_reason # -> "max_turns" | "max_tokens" | "max_seconds"
```

If you ever run code you are unsure about, **start with** max_turns=2. Two turns can't hurt.

Other hazards & habits

- **Develop in mock mode.** starter-code ships a MockClient that returns canned text instantly and for free. Debug your *loop logic* against it; only switch to the real model for real runs. You'll iterate 10× faster and the lab machines stay cool.
- **Set temperature explicitly** (e.g. 0.7) and write it in your transcript. At temperature=0 runs are nearly repeatable, which is great while debugging.
- **Watch the context window.** Small local models hold only a few thousand tokens. When the conversation gets long the model silently forgets the start - that's the Week 3 topic, but notice it early.
- **One model, small.** Use a 3B-class instruct model (llama3.2:3b, qwen2.5:3b). Bigger models are slower and won't make your *dialogue logic* any more correct.
- **Kill switch.** Know ctrl-c. Know how to stop Ollama (ollama stop or quit the app). If a machine is thrashing, stop first, debug second.

If something goes wrong in lab

1. ctrl-c the script.

2. If the machine is still busy, stop the model (`ollama stop`).
3. Re-run with `max_turns=2` in mock mode and find the bug there.

READING

Reading List & References

Reading List & References

Kept short and skimmable on purpose - 2nd-year students read one or two core items per week, not a literature pile. "Core" = expected; "Going further" = optional.

Week 1 - Foundations: an LLM is a message function

Core

- Ollama docs - **Quickstart** and the **Chat API** (`/api/chat`): the messages array of `{role, content}`.
- "What is a token?" - any provider's tokenizer explainer; the idea that text → tokens and models are billed/limited in tokens. (Local models have a fixed **context window**, e.g. ~4k–8k tokens for small models - that limit drives Week 3.)

Going further

- Karpathy, *"Let's build the GPT tokenizer"* (video) - what tokens really are.
- Jurafsky & Martin, **Speech and Language Processing**, ch. on language models (background).

Week 2 - Agents, roles, and the loop

Core

- Anthropic, **Building Effective Agents** - the "agent = model in a loop" framing and why simple beats complex.
- The `system / user / assistant` role convention: re-read the Ollama chat API with the question "whose message is user when two agents talk to each other?"

Going further

- Weng, Lilian - *"LLM Powered Autonomous Agents"* (blog) - components of an agent.

Week 3 - Multi-agent dialogue, memory, and judging

Core

- Du et al., *"Improving Factuality and Reasoning in Language Models through Multiagent Debate"* (2023) - two models arguing improves answers; the canonical multi-agent-debate paper.
- Zheng et al., *"Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena"* (2023) - using an LLM to score other LLMs, and its biases (position bias, verbosity bias) - directly relevant to your judge agent.

Going further

- Park et al., *"Generative Agents: Interactive Simulacra of Human Behavior"* (2023) - memory and reflection in agent societies.
- Liang et al., *"Encouraging Divergent Thinking ... through Multi-Agent Debate"* (2023).

Week 4 - Failure modes, evaluation, and frameworks

Core

- Anthropic, *"Building Effective Agents"* (re-read the "when not to use agents" section).
- Skim **one** framework's intro to see what you re-implemented by hand - pick one:
- AutoGen (Microsoft): conversable agents & group chat.
- LangGraph: agents as a state graph.
- CrewAI: role-based agent crews.

Going further

- Sycophancy in LLMs (e.g. Anthropic, *"Towards Understanding Sycophancy in Language Models"*) - why two agents may just agree with each other (relevant to your failure analysis).

Tools & docs (reference all term)

- **Ollama** - · model library
- **Python** requests / the ollama Python package
- A token-counting note for your model (small Llama/Qwen models use ~0.75 words/token as a rough guide; the Ollama response also returns `exact_prompt_eval_count / eval_count`).

A note on citing AI assistance

You may use an LLM to help write your code and report. State where you did, and remember there might be random group selection for oral presentations if needed. See the Syllabus for details.

DOCUMENT

Course Handbook (PDF)

Attached file: *course-handbook.pdf*

DOCUMENT

Glossary - Agent Communication & Multi-Agent Systems

Attached file: *glossary.pdf*

DISCUSSION

Introduce Yourself & Your Scenario Choice

Welcome to AgentCom! Introduce yourself and share which scenario you're considering for your project (Debate, Collaborative Solver, Interview, Socratic Tutor, or Red vs. Blue). Why does it interest you?